

Level Editor

Operation Guide

F1 Editor Mode System
JSON-Driven Level Configuration

Document Version:	1.0
Last Updated:	2025-11-30
Author:	Struct Squad
Institution:	DigiPen Institute of Technology

1. Overview

The Level Editor is an integrated development tool that allows designers and developers to modify game levels in real-time without recompiling code. Built on a Lua scripting foundation with JSON-driven configuration, the editor provides immediate visual feedback and seamless workflow integration.

1.1 Purpose and Goals

- Enable rapid iteration on level layouts without code compilation
- Provide visual debugging tools through ImGui integration
- Support JSON-driven configuration for designer-friendly workflows
- Maintain separation between gameplay and editor modes
- Facilitate hot-reloading of level configurations

1.2 Supported Level Types

Level Type	Script File	JSON Config	Features
Main Menu	MainMenuLevel.lua	mainmenu_config.json	Background, Logo, Corner sprites, Buttons
Tutorial	TutorialLevel.lua	tutorial_config.json	Scroll overlay, Text-as-buttons, Skip functionality
Level 1-3	Level1.lua, etc.	level1_config.json, etc.	TileMap, Player/Enemy positions, Objectives

1.3 Target Users

Level Designers: Adjust layouts, positions, and visual elements through JSON configuration

Programmers: Debug entity systems, inspect state, and validate game logic

QA Testers: Verify level configurations and identify visual issues

2. Getting Started

2.1 System Requirements

Component	Requirement
Engine Version	Custom C++ Game Engine (2025)
Lua Version	Lua 5.4+ (integrated)
JSON Library	nlohmann/json
Graphics	ImGui-enabled build
Input System	Keyboard support (F1, ESC, Arrow keys)

2.2 Launching the Editor

The editor mode is seamlessly integrated into the game runtime and can be activated at any time during level execution:

1. Start the game application normally
2. Navigate to any supported level (Main Menu, Tutorial, or Gameplay levels)
3. Press **F1** to enter Editor Mode
4. The screen will display "EDITOR MODE" indicator in the top-left corner
5. All interactive buttons will appear grayed out
6. ImGui debug panels will become visible (if enabled)

Note: There is a 0.3-second cooldown between mode toggles to prevent accidental rapid switching. If F1 doesn't respond immediately, wait briefly and try again.

3. Basic Operations

3.1 Mode Switching

The F1 key toggles between Game Mode and Editor Mode:

Mode	State	Characteristics
Game Mode	<code>SetEnginePlayState(true)</code>	Buttons enabled, ImGui hidden, normal gameplay
Editor Mode	<code>SetEnginePlayState(false)</code>	Buttons disabled, ImGui visible, inspection enabled

Code Implementation: The mode toggle is handled in the `OnUpdate()` function of each level script:

```
if IsKeyDown("F1") then
    if editorToggleCooldown <= 0 then
        ToggleEditorMode()
        if IsEditorMode() then
            SetEnginePlayState(false)
        else
            SetEnginePlayState(true)
        end
        editorToggleCooldown = 0.3
    end
end
end
```

4. Keyboard Shortcuts Reference

The following keyboard shortcuts are available in the Level Editor:

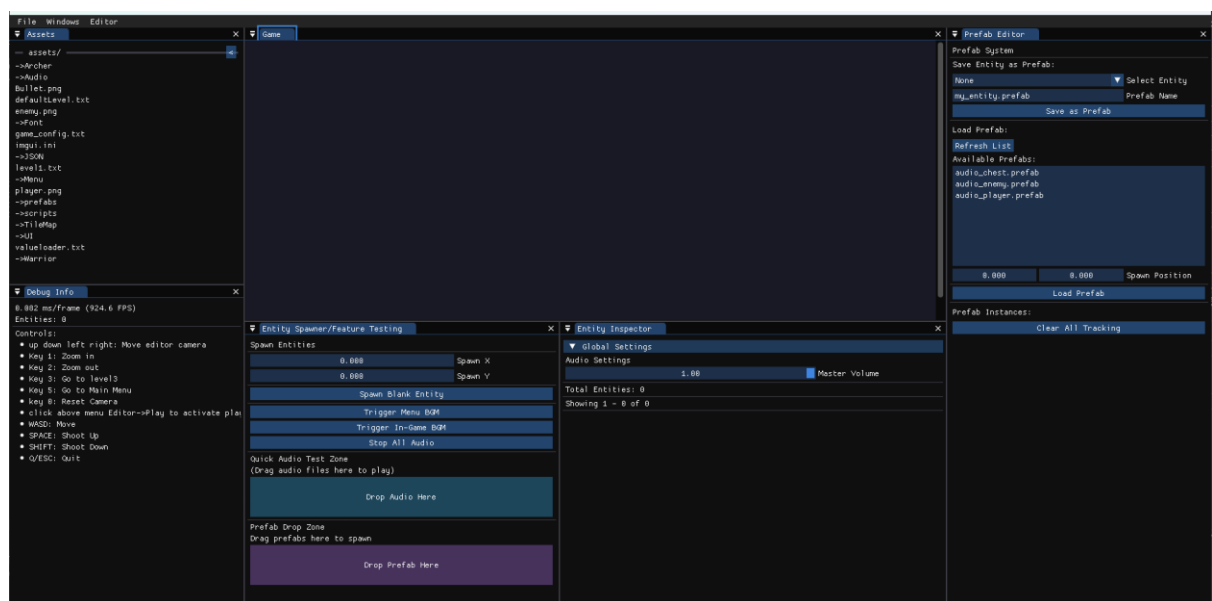
Key	Function	Context	Notes
F1	Toggle Editor Mode	All levels	0.3s cooldown between toggles
ALT+TAB	Switch to full screen	All levels	
Arrow Keys	Edit mode camera control	Demo Level excluded	The Demo Level is set to run in editor mode; the arrow keys are used to control the player.

5. Best Practices

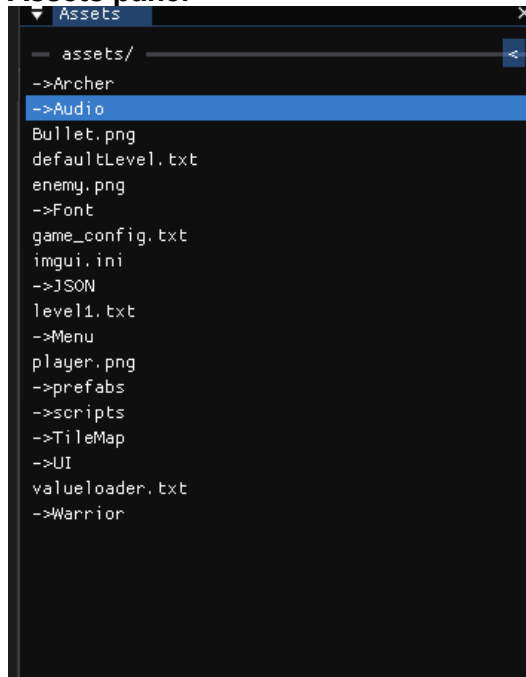
5.1 General Guidelines

- **Always use Editor Mode for layout adjustments:** Toggle to Editor Mode before making mental notes of position changes to avoid accidental button clicks.
- **Validate JSON before testing:** Use a JSON validator (e.g., jsonlint.com) to catch syntax errors before restarting the application.
- **Make incremental changes:** Adjust one property at a time and test immediately. This makes it easier to identify what caused any issues.
- **Keep backups of working configurations:** Before making major changes, copy the working JSON file to a backup (e.g., `mainmenu_config.json.backup`).
- **Use meaningful IDs:** When adding new sprites or buttons, use descriptive IDs like "top_left_corner" instead of "sprite1".
- **Test in both modes:** Always test changes in both Editor Mode (for visual verification) and Game Mode (for functionality).
- **Leverage version control:** Commit working JSON configurations to Git before making experimental changes.

Global screenshot

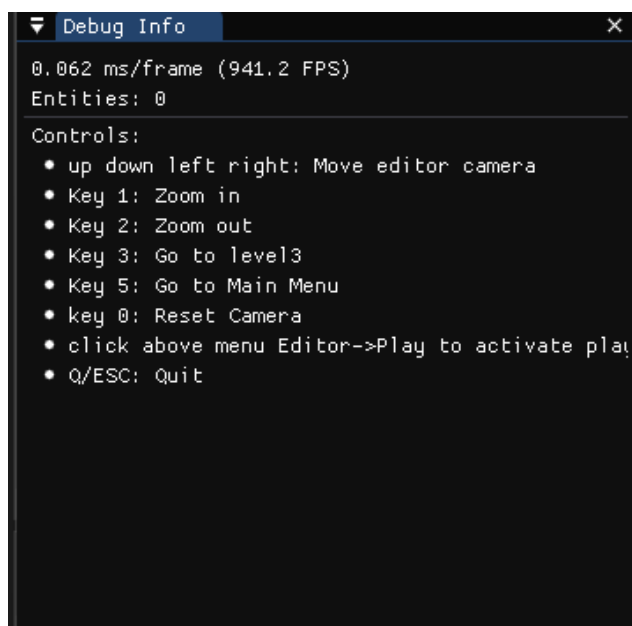


Assets panel



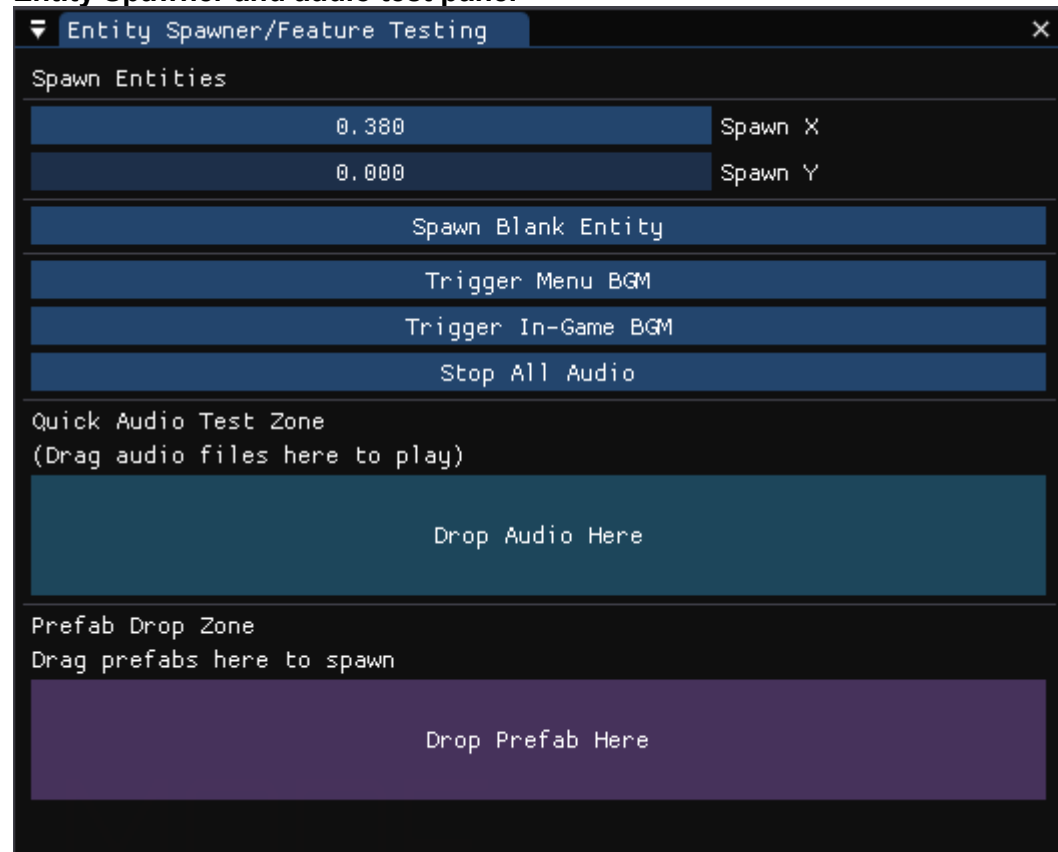
Users can directly generate the corresponding entities by double-clicking the image on the panel.

Debug panel.



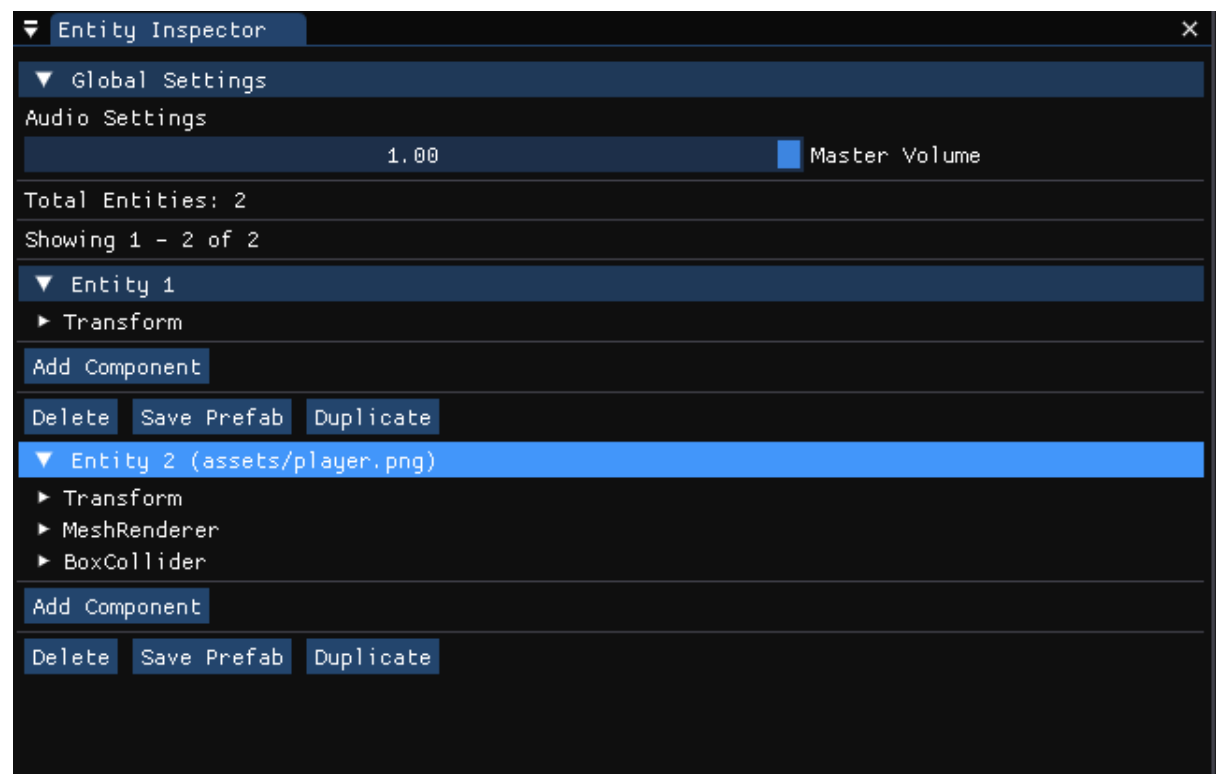
This panel displays the current FPS, the number of entities, and operating guidelines.

Entity Spawner and audio test panel



Users can use this panel to control the location where entities are generated, generate blank entities, and test audio.

Entity Inspector



Users can use this panel to control the position, size, rotation angle, and various component details of entities. Selected entities will be highlighted in blue.

Prefab panel



User can save the selected entity as a prefab through this panel and reload it.

Game panel



This panel is primarily for display; player actions and editing will be shown through it. In edit mode, players can select entities by clicking directly or dragging them to adjust their position. Right-clicking rotates the currently selected entity.

Conclusion

The Level Editor provides a powerful workflow for rapid iteration on game levels through the combination of F1 editor mode toggle, JSON-driven configuration, and ImGui integration. By following the best practices and workflows outlined in this guide, you can efficiently create and modify game levels without the need for code compilation.

Key takeaways:

- F1 toggles Editor Mode for visual inspection and debugging
- JSON files in `assets/JSON/` drive all level configurations
- ImGui provides real-time entity inspection and state debugging
- Button callbacks check `IsEditorMode()` to prevent unintended actions

- Always validate JSON syntax before testing changes
- Use version control to track configuration changes

For questions, bug reports, or feature requests regarding the Level Editor, contact:

GE YONGQI
yongqi.ge@digipen.edu
DigiPen Institute of Technology

Thank you for using the Level Editor. Happy level designing!